MYSQL TRIGGERS – BANK DATABASE

Practical Execution and Results

Student ID: 2520090170

Course: Database Management Systems

Database: BankDB

1. AIM

To create and demonstrate database triggers in MySQL using a Bank Management System and verify that the triggers execute automatically when specified database events occur.

2. SHORT INTRODUCTION

A trigger is a special type of stored procedure that automatically executes (or "fires") in response to specific events on a particular table in a database. Triggers are powerful tools for maintaining data integrity and enforcing business rules at the database level.

Triggers can be defined to fire at different times:

• BEFORE triggers: Execute before the triggering event occurs

• AFTER triggers: Execute after the triggering event occurs

Triggers can respond to different database events:

• INSERT: When a new row is added to a table

• UPDATE: When existing data in a row is modified

• DELETE: When a row is removed from a table

3. DATABASE USED

BankDB

The BankDB database contains several tables including:

• Account: Stores customer account information

• Bank_Transaction: Records all transactions (deposits, withdrawals)

• Transaction_Audit: Maintains audit logs of all transactions

4. TRIGGERS CREATED

CheckBalance

Trigger Timing/Event: BEFORE UPDATE on account table

Purpose: Validates that the account balance does not become negative. If an attempt is made to set the balance below zero, the trigger signals an error and prevents the update.

CheckTransactionAmount

Trigger Timing/Event: BEFORE INSERT on bank_transaction table

Purpose: Ensures that transaction amounts are always greater than zero. Prevents insertion of invalid transactions with zero or negative amounts.

PreventInsufficientWithdrawal

Trigger Timing/Event: BEFORE INSERT on bank_transaction table

Purpose: Validates withdrawal transactions to ensure the account has sufficient balance. Checks if the transaction type is "WITHDRAW" and verifies that the account balance is sufficient to cover the withdrawal amount.

TransactionAudit

Trigger Timing/Event: AFTER INSERT on bank_transaction table

Purpose: Automatically records transaction details into the Transaction_Audit table whenever a new transaction is successfully inserted into the Bank_Transaction table. This creates an audit trail for compliance and monitoring.

UpdateBalanceAfterTransaction

Trigger Timing/Event: AFTER INSERT on bank_transaction table

Purpose: Updates the account balance in the Account table after a transaction is inserted. For DEPOSIT transactions, the balance is increased; this ensures the account balance is always synchronized with transaction records.

5. PRACTICAL EXECUTION / SCREENSHOTS

Screenshot 1: Database Tables

The following screenshot displays the tables in the BankDB database and the structure of the Bank_Transaction table:

```
mysql> show tables;
+-------------------------------------+
| Tables_in_ssvarahaasini_2520090170 |
+-------------------------------------+
| account                             |
| bank_transaction                    |
| class                               |
| class_info                          |
| customer                            |
| customer_transactions               |
| department                          |
| first_table                         |
| loan                                |
| orders                              |
| physician                           |
| second_table                        |
| student_marks                       |
| transaction_audit                   |
+-------------------------------------+
14 rows in set (0.07 sec)

mysql> DESC bank_transaction;
+------------------+---------------+------+-----+-------------------+-------------------+
| Field            | Type          | Null | Key | Default           | Extra             |
+------------------+---------------+------+-----+-------------------+-------------------+
| Transaction_ID   | int           | NO   | PRI | NULL              | auto_increment    |
| Account_No       | int           | YES  | MUL | NULL              |                   |
| Transaction_Type | varchar(20)   | YES  |     | NULL              |                   |
| Amount           | decimal(12,2) | YES  |     | NULL              |                   |
| Transaction_Date | datetime      | YES  |     | CURRENT_TIMESTAMP | DEFAULT_GENERATED |
+------------------+---------------+------+-----+-------------------+-------------------+
5 rows in set (0.03 sec)
```

Figure 1: Database tables in BankDB and Bank_Transaction table structure

Screenshot 2: SHOW TRIGGERS

The following screenshot displays all triggers currently defined in the BankDB database. This output confirms that all five triggers have been successfully created:

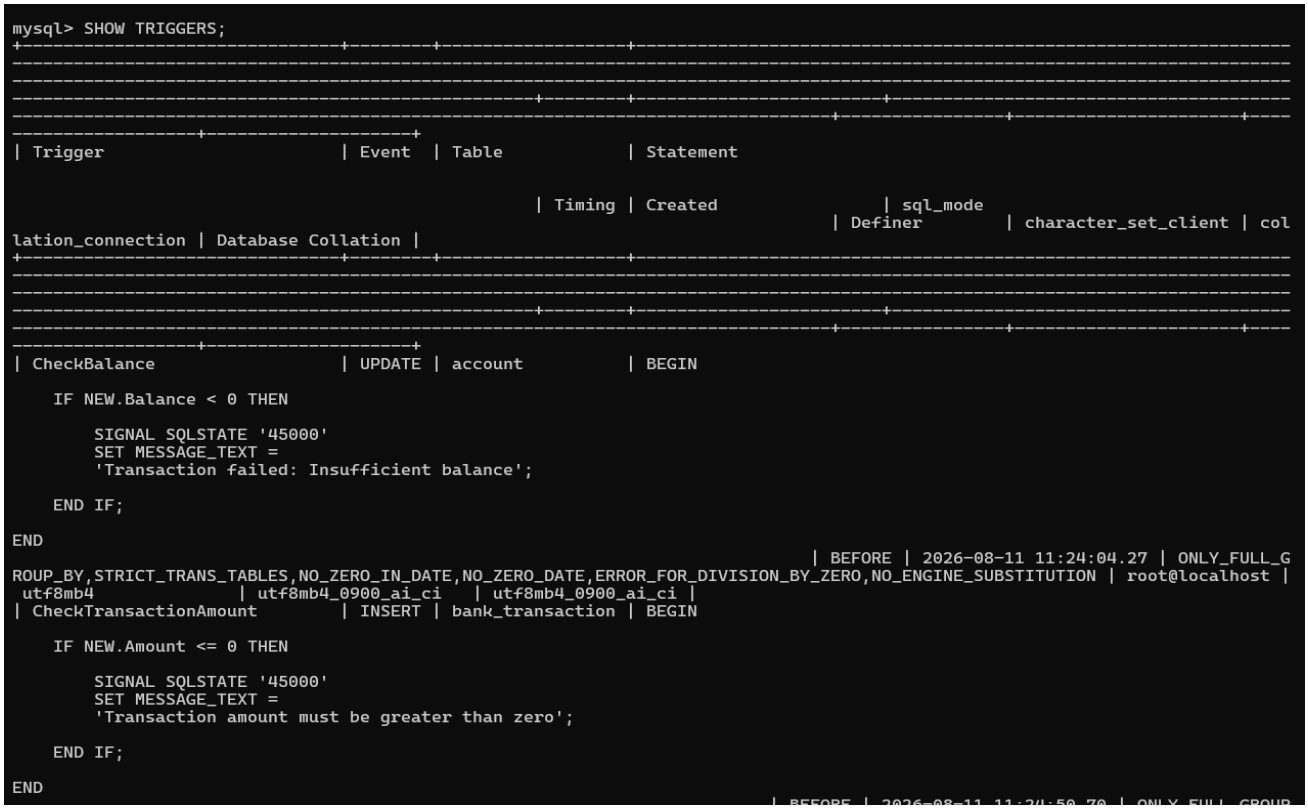


Figure 2: List of all triggers in the BankDB database

Screenshot 3: Trigger Definitions (SHOW CREATE TRIGGER output)

The following screenshot displays the complete definitions of multiple triggers created on the Bank_Transaction table:



Figure 3: Complete SQL definition of triggers including PreventInsufficientWithdrawal, TransactionAudit, and UpdateBalanceAfterTransaction

Screenshot 4: TransactionAudit Trigger Definition

This screenshot displays the complete definition of the TransactionAudit trigger using SHOW CREATE TRIGGER command. This trigger is an AFTER INSERT trigger that automatically inserts transaction details into the Transaction_Audit table:

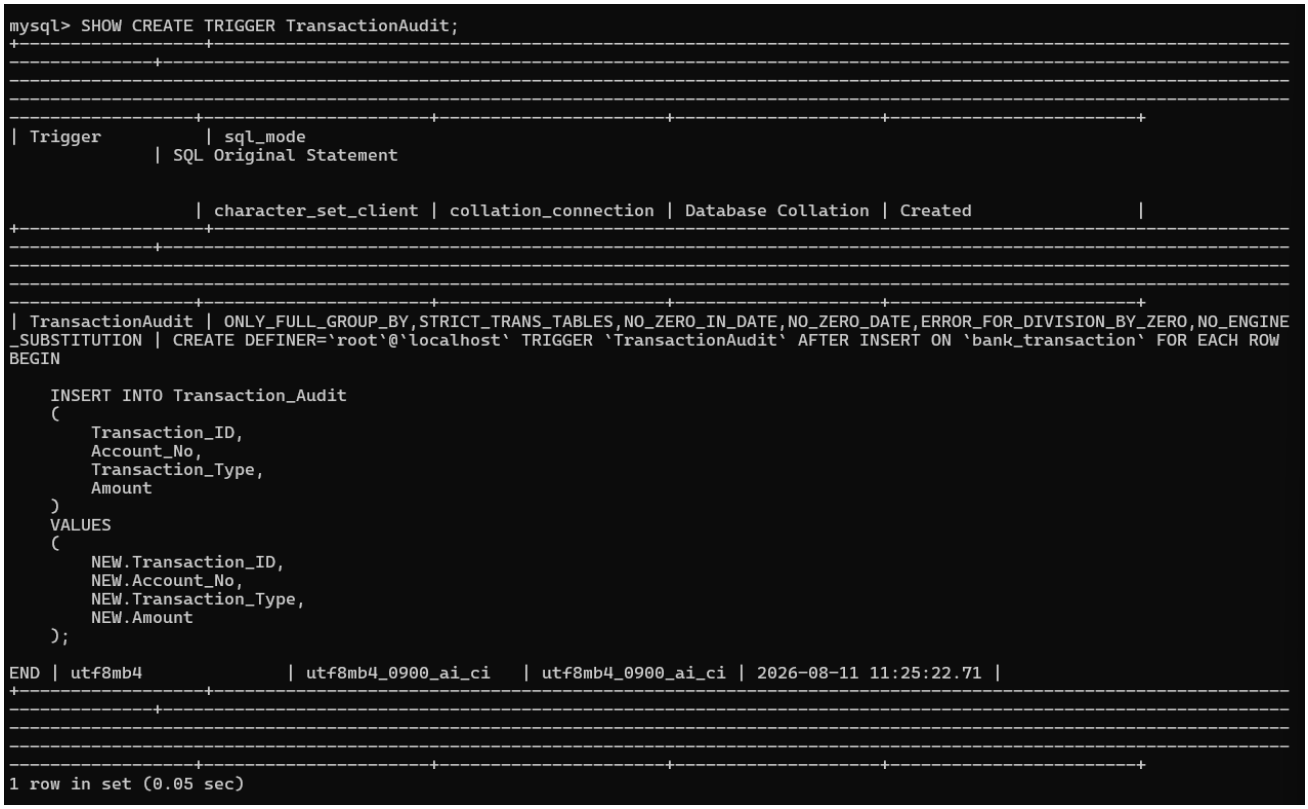


Figure 4: Complete SQL definition of the TransactionAudit trigger

Screenshot 5: Trigger Execution Verification

This screenshot demonstrates the automatic execution of the TransactionAudit trigger. The first SELECT shows the initial audit records. Then, an INSERT statement adds a new transaction to the Bank_Transaction table. The second SELECT shows that the TransactionAudit trigger automatically created a corresponding audit entry (Audit_ID 3) without any manual insertion into the Transaction_Audit table:

```
mysql> SELECT * FROM Transaction_Audit;
+----------+----------------+------------+------------------+---------+---------------------+
| Audit_ID | Transaction_ID | Account_No | Transaction_Type | Amount  | Audit_Date          |
+----------+----------------+------------+------------------+---------+---------------------+
|        1 |              6 |      10001 | DEPOSIT          | 2500.00 | 2026-08-11 11:25:33 |
|        2 |              7 |      10001 | DEPOSIT          | 5000.00 | 2026-08-11 11:26:41 |
+----------+----------------+------------+------------------+---------+---------------------+
2 rows in set (0.02 sec)

mysql> INSERT INTO Bank_Transaction
    -> (Account_No, Transaction_Type, Amount)
    -> VALUES
    -> (10001, 'DEPOSIT', 2500);
Query OK, 1 row affected (0.11 sec)

mysql>
mysql> SELECT * FROM Transaction_Audit;
+----------+----------------+------------+------------------+---------+---------------------+
| Audit_ID | Transaction_ID | Account_No | Transaction_Type | Amount  | Audit_Date          |
+----------+----------------+------------+------------------+---------+---------------------+
|        1 |              6 |      10001 | DEPOSIT          | 2500.00 | 2026-08-11 11:25:33 |
|        2 |              7 |      10001 | DEPOSIT          | 5000.00 | 2026-08-11 11:26:41 |
|        3 |              8 |      10001 | DEPOSIT          | 2500.00 | 2026-08-28 12:26:46 |
+----------+----------------+------------+------------------+---------+---------------------+
3 rows in set (0.00 sec)
```

Figure 5: Verification of TransactionAudit trigger execution - automatic audit record creation on transaction INSERT

6. WORKING OF TRANSACTION AUDIT TRIGGER

The TransactionAudit trigger operates through the following process:

INSERT INTO Bank_Transaction

↓

TransactionAudit Trigger fires automatically

↓

New record inserted into Transaction_Audit

When a new transaction is inserted into the Bank_Transaction table, the TransactionAudit trigger automatically fires after the insert completes. The trigger extracts the transaction details (Transaction_ID, Account_No, Transaction_Type, and Amount) from the newly inserted row and inserts this information into the Transaction_Audit table. This happens without any manual intervention, creating a complete audit trail of all transactions in real-time.

7. TRIGGER SUMMARY TABLE

Trigger Name	Timing / Event	Purpose
CheckBalance	BEFORE UPDATE on account	Validates that balance does not become negative
CheckTransactionAmount	BEFORE INSERT on bank_transaction	Ensures transaction amounts are greater than zero

PreventInsufficientWithdrawal	BEFORE INSERT on bank_transaction	Prevents withdrawal if account has insufficient balance
TransactionAudit	AFTER INSERT on bank_transaction	Automatically logs transaction details to Transaction_Audit table
UpdateBalanceAfterTransaction	AFTER INSERT on bank_transaction	Updates account balance after transaction is recorded

8. RESULT

The MySQL triggers were successfully created and verified in the BankDB database. The triggers automatically perform validation, balance-related operations, and transaction auditing. The TransactionAudit trigger successfully records transaction information in the Transaction_Audit table whenever a new transaction is inserted into the Bank_Transaction table. The automatic execution of these triggers ensures data integrity, prevents invalid transactions, and maintains a complete audit trail of all database operations.